

into your app, after the lines that import all the modules, create constant global objects, etc. The recommended place to put these lines is at the starting of the well-known `if '__main__' == __name__` block in your Python application. Try it out after this, and see if your app seems to run faster.

```
try:
    import psyco
    psyco.full()
except ImportError:
    pass
```

Under Psycho's hood

Let's explore how Psycho works, what performance boosts we can practically expect, and how to achieve the best speed-up with it. Psycho works somewhat like JIT (Just in Time) compilers in other virtual-machine-based programming languages like Java. In fact, Psycho is like a JIT compiler; it compiles chunks of Python byte-code into native machine instructions at runtime. But it does more than a JIT compiler. From a single chunk of Python byte-code, it creates multiple versions of compiled native code, for different kinds of objects.

Psycho profiles the Python program at runtime, and accordingly marks some parts of the program as 'hot zones', for which it emits the native machine instructions. It keeps on using the different versions of generated native-code chunks for the various types of objects used in the hot zones. Thus, Psycho tries to guess the performance-oriented parts of your Python program, and compiles those into native-code chunks to provide performance comparable to C-like native-compiled languages. It also performs various runtime optimisations, to emit the most optimised native-code instructions. The `psyco.full()` function shown above instructs Psycho to tackle the whole Python code that is running, and native-compile as much as possible.

How much, exactly, performance gets boosted by using Psycho, is difficult to guess, due to the complicated mechanism it uses to generate native-compiled versions of your Python programs. However, there are some thumb rules that help you extract the highest performance boosts:

- First, Psycho is most useful for Python programs that are CPU-bound; that is, they do a lot of looping, mathematical calculations, walking lists, string and number operations, etc. Psycho may not be useful to Python code that is I/O-bound (waiting for network events, disk reads, writes or blocking operations), as the performance in those cases is hampered by the GIL, over which Psycho has no control. We will look at ways to overcome the GIL issue, in later sections of the article.
- As mentioned earlier, Psycho generates multiple versions of native-code chunks, optimised for different types of objects—that means Psycho needs a lot of additional memory. Forcing Psycho to work on an entire large Python program that is not a good candidate for Psycho

optimisation will most likely degrade the performance of your program. Therefore, avoid using `psyco.full()`, except in cases where your program is short, and/or you are sure that the whole program is CPU-bound, by the nature of the problem you are solving. You can instruct Psycho to not try to compile the whole Python program, but to first profile the program, and then native-compile only the critical parts based on this profiling. Do this by using `psyco.profile()` instead of `psyco.full()`. Profiling is recommended for large programs. You can also pass a *watermark* value to `psyco.profile()`, which should be between 0.0 and 1.0. A smaller *watermark* indicates that more functions are to be native-compiled, and the larger value that less functions should be compiled. The default *watermark* value is 0.09, if you don't pass the parameter.

- The Psycho functions `psyco.full()` and `psyco.profile()` take another argument, *memory*, which specifies the kilobytes limit below which the functions are compiled. Psycho also provides a `log()` function to enable logging (the log file is named `program.log-psyco`, by default). The following Psycho code enables logging and the profiled compilation of those functions that take at least 7 per cent of execution time, and less than 200 kilobytes per function:

```
psyco.log()
psyco.profile(0.07, memory=200)
```

- You can also manually choose which functions you want Psycho to compile, using the `psyco.bind()` and `psyco.proxy()` functions. Both these functions take the name of the function to be compiled (in Python, that is a reference to a *function object*). The difference in their behaviour is as follows:

The `proxy()` function native-compiles the passed function object, and returns a reference to the native-code-compiled chunk as its return value. The returned reference is still callable; it is still a function object. However, the original function object remains unchanged—the same byte-code that was created when you ran the program. To run the optimised code, you must invoke the native-code-compiled function object via the variable containing the `proxy()` function's return value (*g* in the example below). If you invoke the function by its original name (`function2` as from a `def function2()` statement) then you are calling the byte-code/unoptimised version.

In contrast, `psyco.bind()` compiles the function object passed to it, and replaces the original byte-code function object with the native-code-compiled version. It also native-code-compiles functions called from the passed function. Thus, you can continue to invoke the function with the same name (that is, `function1` as from a `def function1()` statement) and you will be running the optimised native code.

In the sample code snippet shown below, `function1` and the inner functions it calls are native-compiled in-place, but